



PCA in R: Dimensionality Reduction and Component Selection Explained

Why reduce dimensionality?

The Curse of Dimensionality

As dimensionality increases, data becomes increasingly sparse. The density and distance between points lose meaning, which is critical for clustering and outlier analysis. The possible combinations of subspaces grow exponentially. It is in this context that dimensionality reduction becomes essential:

The main approaches to dealing with this problem include:

Principal Component Analysis

Projection that captures the greatest variation in the data

Feature Creation

Construction of new informative attributes

Benefits of Reduction

- Avoids the curse of dimensionality
- Eliminates irrelevant features and reduces noise
- Decreases time and space required in mining
- Enables easier visualization of data

Supervised Techniques

Feature selection with feedback

Regression Models

Identification of essential linear relationships

📖 **Reference:** M. Dash, H. Liu, and J. Yao, 1997, Dimensionality reduction of unsupervised data, Proceedings of the International Conference on Tools with Artificial Intelligence.

Principal Component Analysis (PCA)

PCA is an **unsupervised** technique that finds new axes (principal components) capturing the directions of **maximum variance**, projecting data into a lower-dimensional space while preserving as much information as possible.

1. Intuition

PCA identifies the directions along which data varies the most — the axes that carry the most information.

2. Mathematical Foundation

PCA computes the **covariance matrix**, then performs **eigendecomposition** to find principal component directions and their explained variance.

3. Explained Variance

Eigenvalues are sorted in descending order to guide how many components to keep — typically retaining 90–95% of total variance.

The eigenvectors of the covariance matrix define the new feature space, ordered by their ability to explain the total variance of the data.

📄 **References:** H. Abdi and L.J. Williams, 2010, Principal component analysis, Wiley Interdisciplinary Reviews: Computational Statistics, v. 2, n. 4, p. 433–459. | Jolliffe, I.T. (2002). *Principal Component Analysis*. 2nd ed. Springer.

Steps of Principal Component Analysis

With the intuition established, we formalize the problem: Given n data vectors of n dimensions, find $k \leq n$ orthogonal vectors (principal components) that can best be used to represent the data.

Data Normalization

Each attribute is adjusted to fall within the same range, ensuring that no variable dominates the analysis solely due to its scale.

Sorting by Importance

The principal components are ordered by decreasing "significance" or strength, measured by the variance they explain.

With these steps defined, we can visualize how the projection occurs in practice.

Computing the Components

Calculate k orthonormal (unit) vectors — the principal components. Each input vector is a linear combination of these k vectors.

Dimensionality Reduction

The data size can be reduced by eliminating weak components (low variance). Using the strongest components, it is possible to reconstruct a good approximation of the original data.

❏ Important: PCA works **only with numerical data**. Categorical variables must be handled separately or encoded appropriately.

❏ **Reference:** H. Abdi and L.J. Williams, 2010, Principal component analysis, Wiley Interdisciplinary Reviews: Computational Statistics, v. 2, n. 4, p. 433–459.

Projection with PCA

Visualizing the process described earlier: The PCA transformation projects high-dimensional data into a lower-dimensional subspace, preserving the structure of maximum variance.

Original Data

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2
5.4	3.9	1.7	0.4

Two-dimensional dataset showing the original distribution with correlation between variables.

PCA Transformation

	PC1	PC2
Sepal.Length	0.5210659	-0.37741762
Sepal.Width	-0.2693474	-0.92329566
Petal.Length	0.5804131	-0.02449161
Petal.Width	0.5648565	-0.06694199

Identification of the principal axes (PC1 and PC2) that capture the maximum variance.

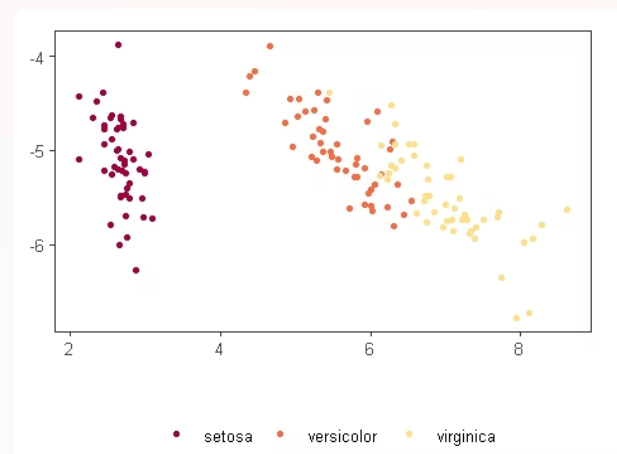
Projected Data

PC1	PC2	Species
2.640270	-5.204041	setosa
2.670730	-4.666910	setosa
2.454606	-4.773636	setosa
2.545517	-4.648463	setosa
2.561228	-5.258629	setosa
2.975946	-5.707321	setosa

Final representation in the new coordinate system defined by the principal components.

The visualization above demonstrates how PCA rotates the feature space to align with the directions of maximum variance, facilitating dimensionality reduction without significant loss of information. In the next section, we will see how to apply PCA in practice using the daltoolbox package.

 **References:** A. Tharwat et al., 2017, Linear discriminant analysis: A detailed tutorial, AI Communications, v. 30, n. 2. | R-bloggers: Computing and Visualizing LDA



Overview – Dimensionality Reduction

The daltoolbox package provides three functions that cover the complete cycle of dimensionality reduction — from projection to automatic selection of the ideal number of components. Below is an overview of each:

dt_pca

Principal Component Analysis. Projects correlated variables onto orthogonal components ordered by explained variance. The number of components can be defined manually or by a curvature heuristic.

fit_curvature_min

Detects the minimum curvature point (knee) in increasing curves, such as error curves. Useful for automatically determining the ideal number of components in minimization scenarios.

fit_curvature_max

Detects the maximum curvature point (elbow) in decreasing curves, such as cumulative explained variance. Indicates where the marginal gain of adding more components begins to diminish.

📄 **General principle:** PCA is unsupervised — it does not use the class column. The curvature heuristics automate the choice of the number of components, avoiding arbitrary decisions.

PCA – Principal Component Analysis

We start with the core function: `dt_pca`, which encapsulates the entire PCA pipeline with support for automatic and manual modes.

```
library(daltoolbox)

iris <- datasets::iris

# Automatic adjustment (elbow heuristic)
mypca <- dt_pca("Species")
mypca <- fit(mypca, datasets::iris)
iris.pca <- transform(mypca, iris)

print(head(iris.pca))
##      PC1      PC2 Species
## 1 2.640270 -5.204041 setosa
## 2 2.670730 -4.666910 setosa
```

```
print(head(mypca$pca.transf))
##           PC1      PC2
## Sepal.Length 0.5210659 -0.37741762
## Sepal.Width -0.2693474 -0.92329566
## Petal.Length 0.5804131 -0.02449161
## Petal.Width 0.5648565 -0.06694199

# Manual definition: 3 components
mypca <- dt_pca("Species", 3)
mypca <- fit(mypca, datasets::iris)
iris.pca <- transform(mypca, iris)
## Result: PC1, PC2, PC3 + Species
```

What it is

Projects correlated variables into orthogonal components ordered by explained variance (SVD/eigendecomposition).

Automatic mode

Number of components chosen by the minimum curvature heuristic on the cumulative variance curve.

Manual mode

`dt_pca("Species", 3)` forces 3 components, adding PC3 to the result.

Loadings

`mypca$pca.transf` shows the contribution of each original variable to each component.

📌 **Key concept:** PCA maximizes the variance captured by each component. Components are orthogonal — with no correlation between them. Ref.: Pearson (1901); Hotelling (1933). Next, we will see how to automatically choose the number of components with `fit_curvature_min`.

Choosing the Number of Components – Knee/Elbow Detection

After applying PCA, a critical decision is determining how many principal components to retain. Arbitrary choices like "keep 95% of variance" are subjective and lack statistical rigor. A more principled approach involves detecting the natural "knee" or "elbow" point in the explained variance curve, which signifies an optimal balance between dimensionality reduction and information preservation.

The Problem of Component Selection

After applying PCA, how do we decide how many components to keep? Arbitrary choices (e.g., "keep 95% of variance") are subjective. A more principled approach is to detect the natural "knee" or "elbow" of the curve.

When to Use `fit_curvature_min`

Use it on **increasing curves** (e.g., cumulative explained variance). The knee marks the point where adding more components yields diminishing returns.

Curvature Analysis

The idea is to compute the mathematical curvature of the curve (related to the second derivative). The point of minimum curvature on an increasing curve corresponds to where the rate of gain starts to diminish — the "knee" point.

Conceptual Description of the Knee

Imagine a curve that rises steeply at first, then flattens. The knee is the transition point between the steep initial rise and the subsequent flatter section, indicating a significant change in the curve's slope.

📖 **Reference:** Satopaa, V. et al. (2011). Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. 31st International Conference on Distributed Computing Systems Workshops (ICDCSW). IEEE.

Minimum Curvature – `fit_curvature_min`

Once PCA is applied, the next question is: how many components should be retained? `fit_curvature_min` answers this automatically for increasing curves, such as cumulative variance.

```
library(daltoolbox)
library(ggplot2)
library(dplyr)

# Loads wine dataset and applies PCA
wine <- get(load(url("https://raw.githubusercontent.com/
  cefet-rj-dal/daltoolbox/main/
  examples/wine.RData")))
pca_res <- prcomp(wine[,2:ncol(wine)],
  center=TRUE, scale.=TRUE)

# Cumulative variance
y <- cumsum(
  pca_res$sdev^2 / sum(pca_res$sdev^2)
)
x <- 1:length(y)
```

```
# Cumulative variance by dimension:
# dim 1: 0.360 | dim 2: 0.552
# dim 3: 0.664 | dim 4: 0.735 ...

# Detects elbow point (increasing curve)
myfit <- fit_curvature_min()
res <- transform(myfit, y)
## x=6, y=0.851 → optimal point

# Plots with vertical line at elbow
plot(grf + geom_vline(
  xintercept = res$x,
  linetype="dashed", color="red"
))
```

Context

The cumulative variance curve from PCA is increasing: the first dimensions concentrate high variance, while marginal gains decrease.

What it does

Computes curvature via the second derivative of a smoothed spline and returns the position of **minimum curvature**.

Result

Dimension 6 identified as the optimal point (cumulative variance ≈ 0.851) — a trade-off between completeness and parsimony.

Visualization

A dashed red line marks the elbow on the cumulative variance chart.

❏ **Key concept:** Use `fit_curvature_min` for **increasing** curves. The minimum curvature point indicates where additional gains become marginal. Ref.: Satopaa et al. (2011). For decreasing curves, the complementary function is `fit_curvature_max` — presented next.

Elbow Detection on Decreasing Curves

The Problem with Decreasing Curves

Many real-world curves are decreasing (e.g., reconstruction error, loss as a function of model complexity). On these curves, we want to find the "elbow" — the point where further reduction yields little additional benefit.

Curvature Analysis: Maximum Curvature

The mathematical curvature is computed via the second derivative of a smoothing spline. On a decreasing curve, the point of **maximum curvature** corresponds to the sharpest bend — the elbow.

Practical Applications

This method is useful for choosing the number of clusters (elbow method in k-means), selecting model complexity, and determining the number of PCA components based on reconstruction error.

Comparing Curvature Detection Functions

Function	Curve Type	What it Finds
fit_curvature_min	Increasing curves (e.g., cumulative variance)	The "knee" point (where marginal gains diminish)
fit_curvature_max	Decreasing curves (e.g., error, loss)	The "elbow" point (where further reduction yields little benefit)

 **Reference:** Satopaa, V. et al. (2011). Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. 31st International Conference on Distributed Computing Systems Workshops (ICDCSW). IEEE.

Maximum Curvature

While `fit_curvature_min` operates on increasing curves, `fit_curvature_max` is its counterpart for decreasing curves — such as error or loss curves.

```
library(daltoolbox)
library(ggplot2)
library(dplyr)
# Synthetic decreasing curve (-log)
x <- seq(from=1, to=10, by=0.5)
dat <- data.frame(
  x = x,
  value = -log(x),
  variable = as.factor("log")
)
grf <- plot_scatter(
  dat,
  label_x = "x",
  label_y = "y",
  colors = "black"
) + theme(text = element_text(size=16))
```

```
# Detects elbow (decreasing curve)
myfit <- fit_curvature_max()
res <- transform(myfit, dat$value)
res
## x      y      yfit
## 1 9 -1.609438 9.224359e-08

# Plots with vertical line at elbow
plot(grf + geom_vline(
  xintercept = res$x,
  linetype="dashed", color="red",
  size=0.5
))
```

Context

Decreasing curves (e.g., error vs. complexity) require identifying the point where further reduction yields little benefit.

What it does

Computes curvature via the second derivative of a smoothing spline and returns the position of **maximum curvature** (elbow).

Result

Position $x=9$ identified as the elbow in the `-log(x)` curve — the point where the curve bends most sharply.

Difference from min

`fit_curvature_min` → increasing curves.

`fit_curvature_max` → decreasing or concave curves.

📌 **Key concept:** Use `fit_curvature_max` for **decreasing** curves. The elbow marks the point where additional reductions no longer compensate. Ref.: Satopaa et al. (2011). With the three functions presented, we have a complete dimensionality reduction pipeline — from projection to automatic component selection.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

